

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 - Redes

Sección 10

Ing. Jorge Yass



Laboratorio #2: Esquemas de
detección y corrección de
errores

Reporte grupal

Fernando Andree Hernández Martínez 23645

Fernando José Rueda Rodas 23748

GUATEMALA

2 de agosto de 2026

1. Descripción de la práctica

En esta práctica se implementó una arquitectura de capas (Aplicación, Presentación, Enlace, Ruido y Transmisión) para la comunicación entre un cajero automático (emisor, en Python) y el servidor de una entidad financiera (receptor, en Node.js), a través de un canal no confiable simulado. En la capa de Enlace se integraron dos algoritmos: **Hamming (12,8)** para corrección de errores y **CRC-32** (polinomio estándar IEEE 802.3, $0x04C11DB7$) para detección de errores. El ruido se simula volteando cada bit de la trama con una probabilidad configurable, y la transmisión se realiza mediante sockets TCP con mensajes JSON delimitados por salto de línea, lo que permite la interoperabilidad entre ambos lenguajes.

2. Metodología de pruebas

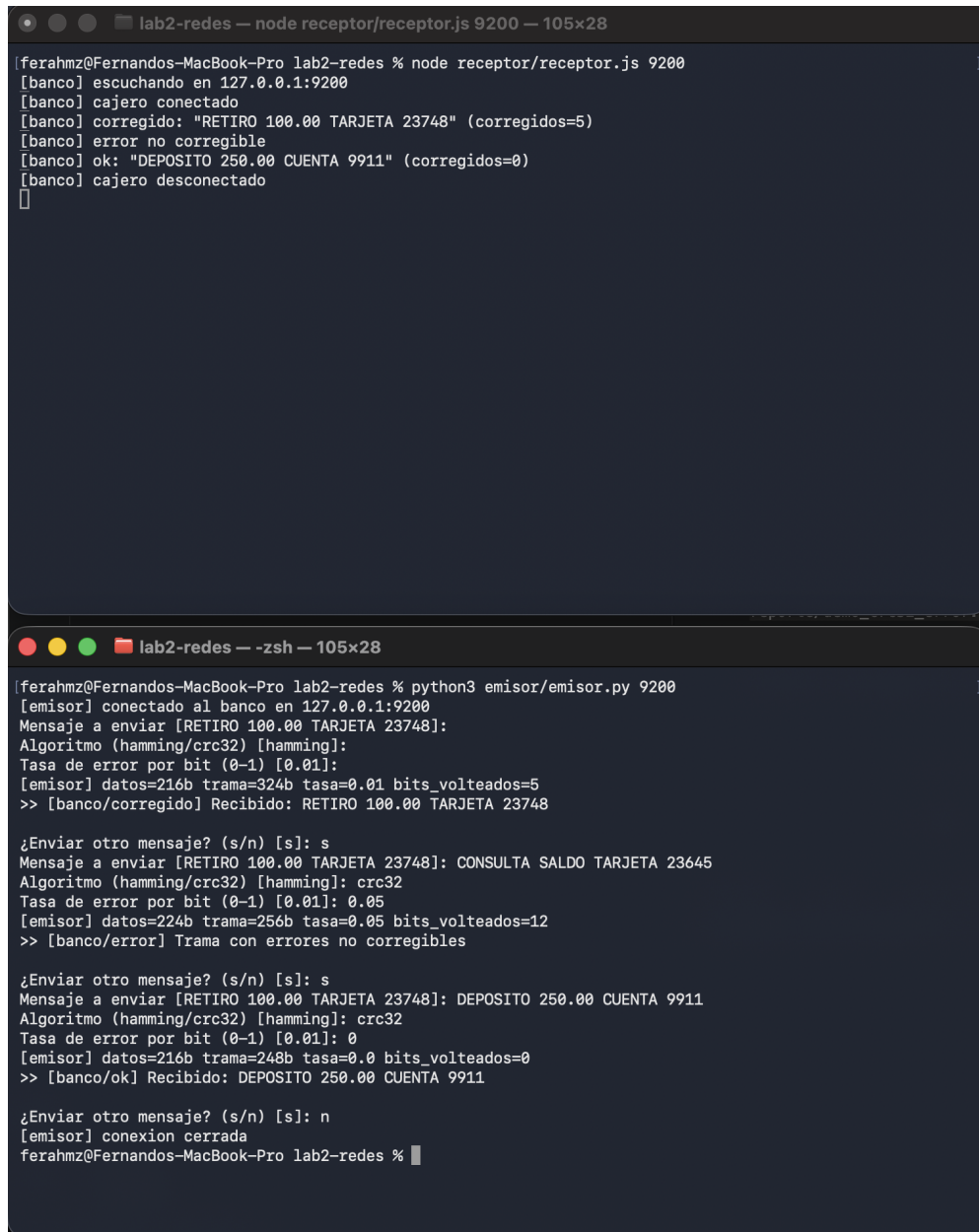
Las pruebas se automatizaron con un *harness* (`pruebas/benchmark.py`) que ejecuta la arquitectura completa de extremo a extremo: por cada envío se genera un mensaje aleatorio, se codifica, se calcula la integridad, se aplica ruido y se transmite por sockets al receptor en Node, clasificando la respuesta del banco en tres resultados posibles:

- **Éxito:** el mensaje entregado coincide con el original.
- **Error detectado:** el algoritmo detectó errores que no pudo (o no puede) corregir y el mensaje fue rechazado.
- **Error no detectado:** el banco aceptó un mensaje corrupto.

Se probaron todas las combinaciones de: algoritmo (Hamming, CRC-32), tamaño de mensaje (4, 16, 64 y 256 caracteres) y probabilidad de error por bit (10^{-3} a 10^{-1}), con 200 repeticiones por combinación (11 200 envíos en total, semilla fija para reproducibilidad).

3. Resultados

3.1. Demostración de la aplicación



```
lab2-redes — node receptor/receptor.js 9200 — 105x28
[ferahmz@Fernandos-MacBook-Pro lab2-redes % node receptor/receptor.js 9200
[banco] escuchando en 127.0.0.1:9200
[banco] cajero conectado
[banco] corregido: "RETIRO 100.00 TARJETA 23748" (corregidos=5)
[banco] error no corregible
[banco] ok: "DEPOSITO 250.00 CUENTA 9911" (corregidos=0)
[banco] cajero desconectado
]
```

```
lab2-redes — -zsh — 105x28
[ferahmz@Fernandos-MacBook-Pro lab2-redes % python3 emisor/emisor.py 9200
[emisor] conectado al banco en 127.0.0.1:9200
Mensaje a enviar [RETIRO 100.00 TARJETA 23748]:
Algoritmo (hamming/crc32) [hamming]:
Tasa de error por bit (0-1) [0.01]:
[emisor] datos=216b trama=324b tasa=0.01 bits_volteados=5
>> [banco/corregido] Recibido: RETIRO 100.00 TARJETA 23748

¿Enviar otro mensaje? (s/n) [s]: s
Mensaje a enviar [RETIRO 100.00 TARJETA 23748]: CONSULTA SALDO TARJETA 23645
Algoritmo (hamming/crc32) [hamming]: crc32
Tasa de error por bit (0-1) [0.01]: 0.05
[emisor] datos=224b trama=256b tasa=0.05 bits_volteados=12
>> [banco/error] Trama con errores no corregibles

¿Enviar otro mensaje? (s/n) [s]: s
Mensaje a enviar [RETIRO 100.00 TARJETA 23748]: DEPOSITO 250.00 CUENTA 9911
Algoritmo (hamming/crc32) [hamming]: crc32
Tasa de error por bit (0-1) [0.01]: 0
[emisor] datos=216b trama=248b tasa=0.0 bits_volteados=0
>> [banco/ok] Recibido: DEPOSITO 250.00 CUENTA 9911

¿Enviar otro mensaje? (s/n) [s]: n
[emisor] conexion cerrada
ferahmz@Fernandos-MacBook-Pro lab2-redes %
```

Figura 1: Intercambio de mensajes entre el cajero (Python, abajo) y el servidor bancario (Node.js, arriba) mediante sockets TCP. Se observan los tres resultados posibles: con Hamming y $p = 0.01$ el ruido volteó 5 bits y los 5 fueron corregidos (*corregido*); con CRC-32 y $p = 0.05$ se voltearon 12 bits y la trama fue rechazada (*error*); con el canal limpio el mensaje se entregó intacto (*ok*).

3.2. Pruebas automatizadas

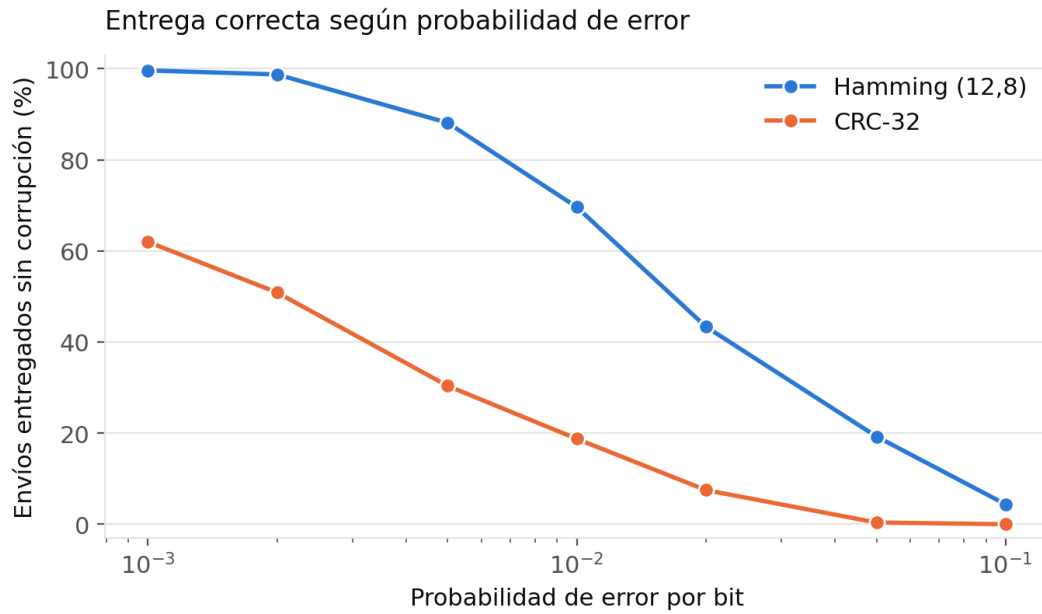


Figura 2: Porcentaje de envíos entregados sin corrupción según la probabilidad de error por bit. Hamming mantiene la entrega gracias a la corrección; CRC-32 solo entrega tramas que llegaron intactas.

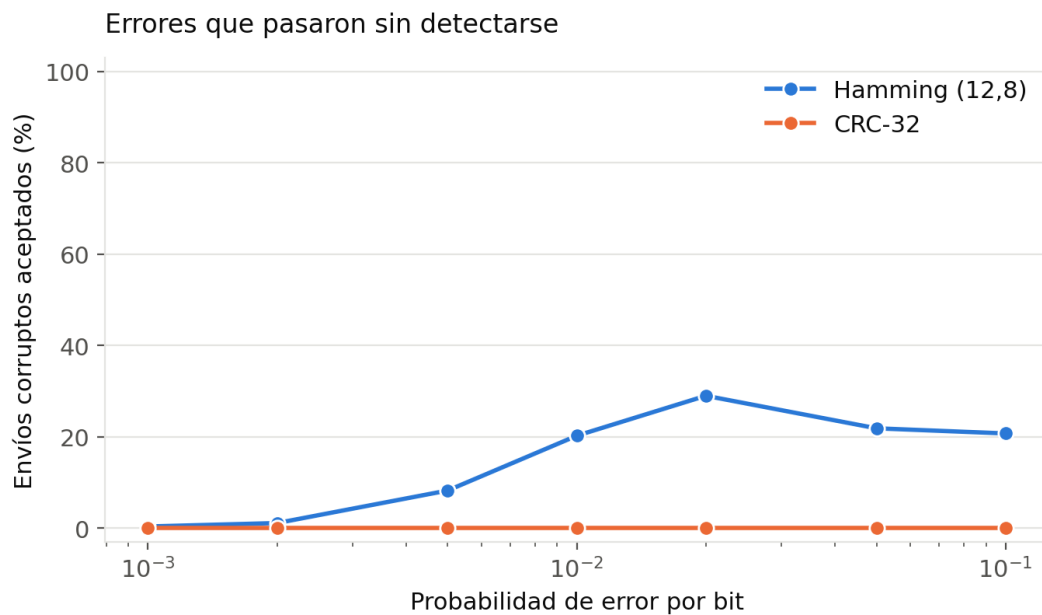


Figura 3: Porcentaje de envíos corruptos aceptados como válidos. CRC-32 no dejó pasar ningún error en las pruebas; Hamming “corrige mal” cuando hay más de un error por bloque.

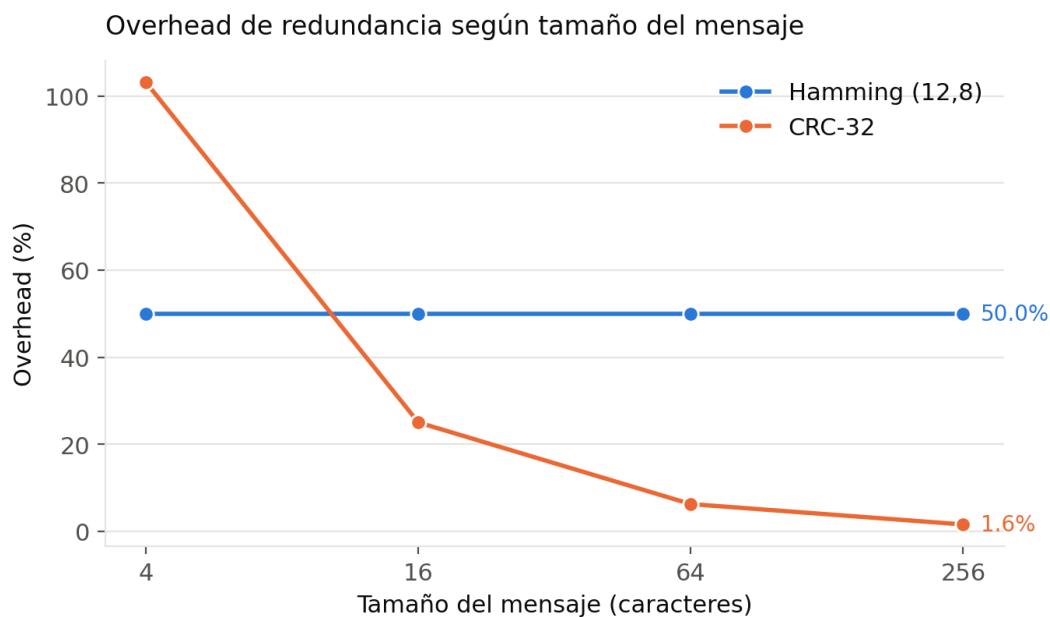


Figura 4: Overhead de redundancia según el tamaño del mensaje. Hamming (12,8) agrega un 50 % constante; el CRC-32 fijo de 32 bits se diluye al crecer el mensaje.

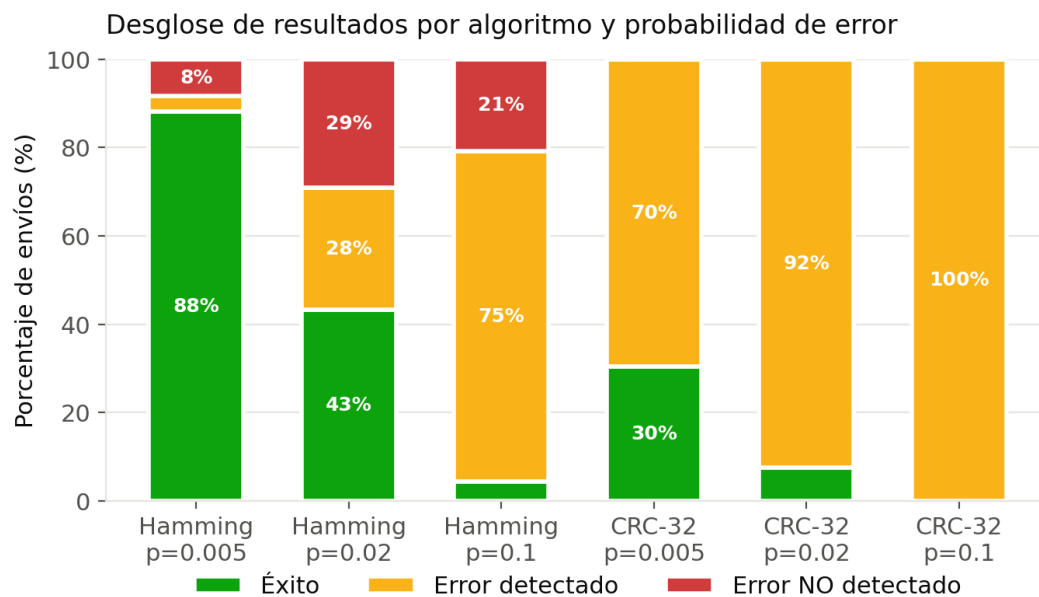


Figura 5: Desglose de resultados por algoritmo para tres probabilidades de error representativas.

4. Discusión

¿Qué algoritmo tuvo un mejor funcionamiento?

Depende del criterio con que se mida. Si el objetivo es *entregar* el mensaje a pesar del ruido, Hamming fue claramente superior (figura 2): con $p = 0.001$ entregó el 99.6 % de los mensajes

correctamente y todavía a $p = 0.01$ mantuvo un 69.6 % de entrega, mientras que CRC-32, al no poder corregir, cayó del 62.0 % al 18.8 % en el mismo rango —CRC-32 solo entrega las tramas que llegaron intactas por azar.

Sin embargo, si el criterio es la *confiabilidad* de lo que se acepta, el resultado se invierte (figura 3): CRC-32 no dejó pasar ni un solo mensaje corrupto en los 5 600 envíos de las pruebas, mientras que Hamming aceptó mensajes corruptos como válidos hasta en un 29.0 % de los envíos ($p = 0.02$). Esto ocurre porque cuando un bloque de 12 bits sufre dos o más errores, el síndrome apunta a una posición equivocada y el decodificador “corrige” un bit que estaba bien, corrompiendo el bloque sin detectarlo. Para una aplicación bancaria, aceptar un retiro alterado es mucho más grave que rechazarlo y retransmitir.

¿Qué algoritmo es más flexible ante mayores tasas de error?

Ninguno de los dos *entrega* mensajes con tasas altas, pero se degradan de forma muy distinta. Hamming tolera bien el ruido mientras los errores caen aislados (a lo sumo uno por bloque de 12 bits), lo que es probable con $p \lesssim 0.005$; más allá, la probabilidad de dos errores en un mismo bloque crece y su tasa de errores no detectados se estanca alrededor del 20–30 %, que es su peor característica. CRC-32 es más “flexible” en el sentido de que su capacidad de detección no se degrada: del rango completo probado (10^{-3} a 10^{-1}) detectó el 100 % de las tramas corruptas (figura 5); simplemente rechaza casi todo cuando el canal es muy ruidoso (99.6 % a $p = 0.05$), delegando la recuperación a una retransmisión.

¿Cuándo conviene detección en lugar de corrección?

La detección (CRC-32) conviene cuando existe un canal de retorno que permita retransmitir —como en esta práctica, donde el cajero y el banco mantienen una conexión TCP bidireccional— y cuando el costo de aceptar datos corruptos es alto. Además su overhead es fijo (32 bits) y se diluye con el tamaño del mensaje: del 14.3 % en mensajes de 4 caracteres al 1.6 % en mensajes de 256 (figura 4). La corrección (Hamming) conviene cuando retransmitir es caro o imposible (enlaces satelitales, streaming en tiempo real, memorias) y se prefiere pagar un overhead constante del 50 % a cambio de recuperar el mensaje sin ida y vuelta. En un escenario real ambos se complementan: corrección para los errores frecuentes y detección como red de seguridad final.

5. Conclusiones

- La arquitectura de capas permitió separar responsabilidades con claridad: fue posible implementar el emisor en Python y el receptor en Node.js, y agregar un segundo algoritmo a la capa de Enlace sin tocar las demás capas, evidenciando el valor del modelo de servicios visto en clase.
- Hamming (12,8) maximiza la entrega en canales con ruido moderado (99.6 % de entrega a $p = 0.001$), pero su garantía se rompe con más de un error por bloque: en las pruebas llegó a aceptar hasta un 29 % de mensajes corruptos sin detectarlos, lo que lo hace inadecuado como único mecanismo de integridad en aplicaciones críticas.
- CRC-32 detectó el 100 % de las tramas corruptas en los 5 600 envíos de prueba y su overhead fijo de 32 bits se vuelve despreciable al crecer el mensaje (1.6 % a 256 caracteres), confirmando por qué es el estándar de integridad en tramas Ethernet.

- La elección entre detección y corrección no es de “mejor o peor” sino de contexto: con un canal de retorno disponible conviene detectar y retransmitir; sin él, o con retransmisiones costosas, conviene pagar el overhead de la corrección.
- Simular el ruido como probabilidad de volteo por bit permitió cuantificar el comportamiento de ambos algoritmos de forma reproducible (11 200 envíos automatizados con semilla fija), en lugar de depender de pruebas manuales aisladas.

6. Citas y referencias

- Tanenbaum, A. S. & Wetherall, D. J. (2011). *Computer Networks* (5.^a ed.). Pearson. Capítulo 3: The Data Link Layer.
- IEEE 802.3. Polinomio estándar CRC-32 ($0x04C11DB7$).